

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Факультет безопасности информационных технологий

Дисциплина:

“Программирование”

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №1

Вариант 1-5

Выполнила:

Студентка гр. 3148 Жук Анастасия



Проверил:

Волков А. Г.

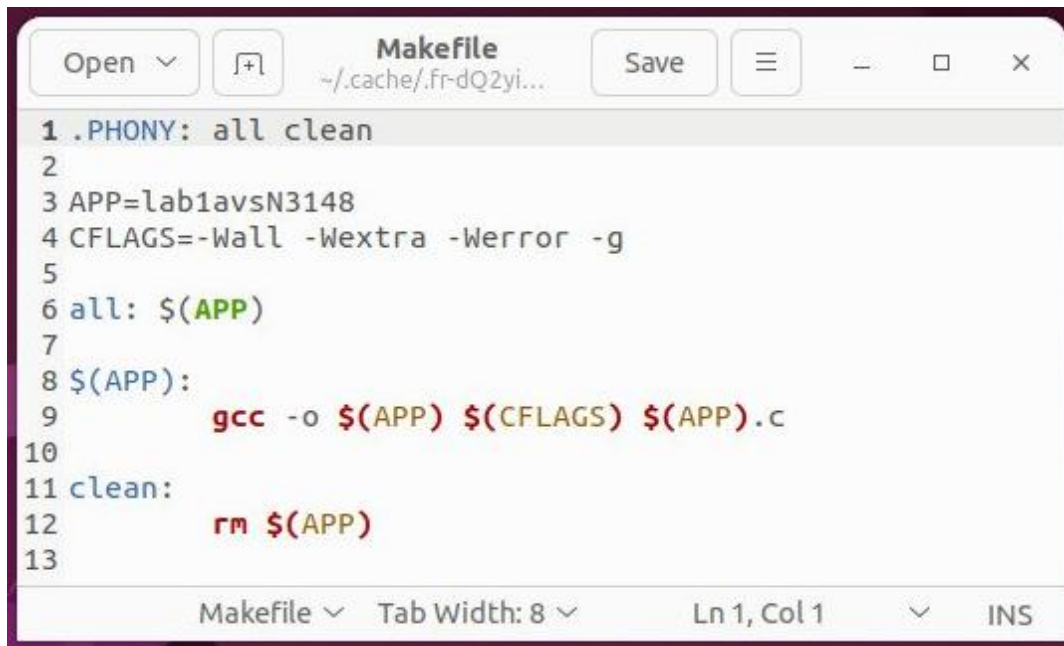
Санкт-Петербург
2023г.

Задание

Разработать на языке C для ОС Linux программу, которая получает число типа unsigned short, выводит его двоичное представление на экран (необходимо выводить все разряды числа, байты числа разделяются пробелами), выполняет преобразование, затем выводит на экран двоичное представление результата преобразования.

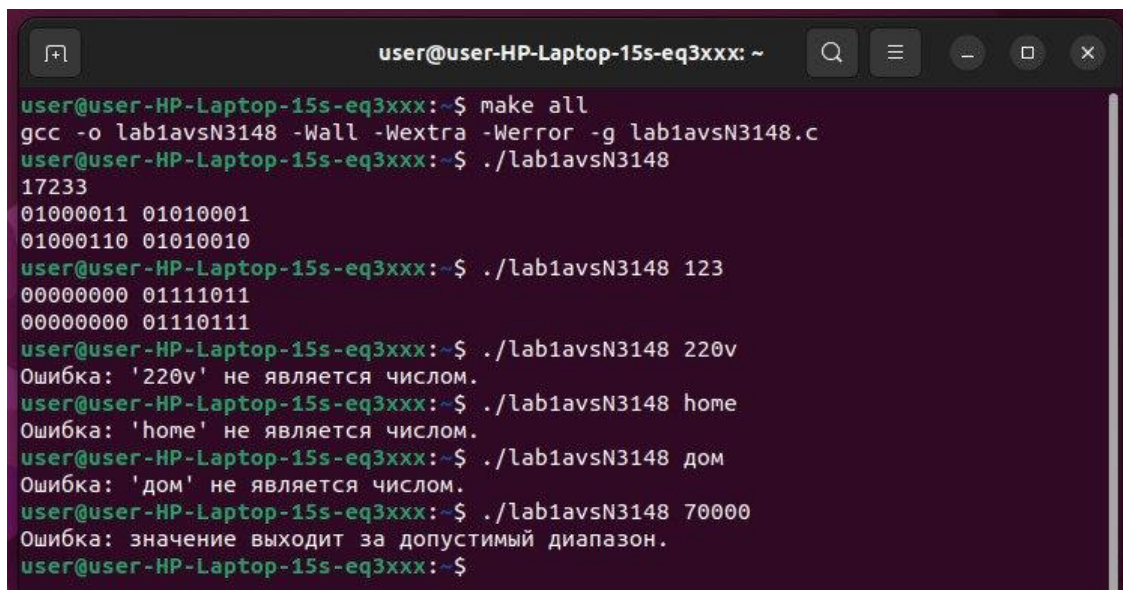
Преобразование: каждую младшую тетраду каждого байта сдвинуть циклически влево на число, содержащееся в двух старших битах старшей тетрады.

MakeFile



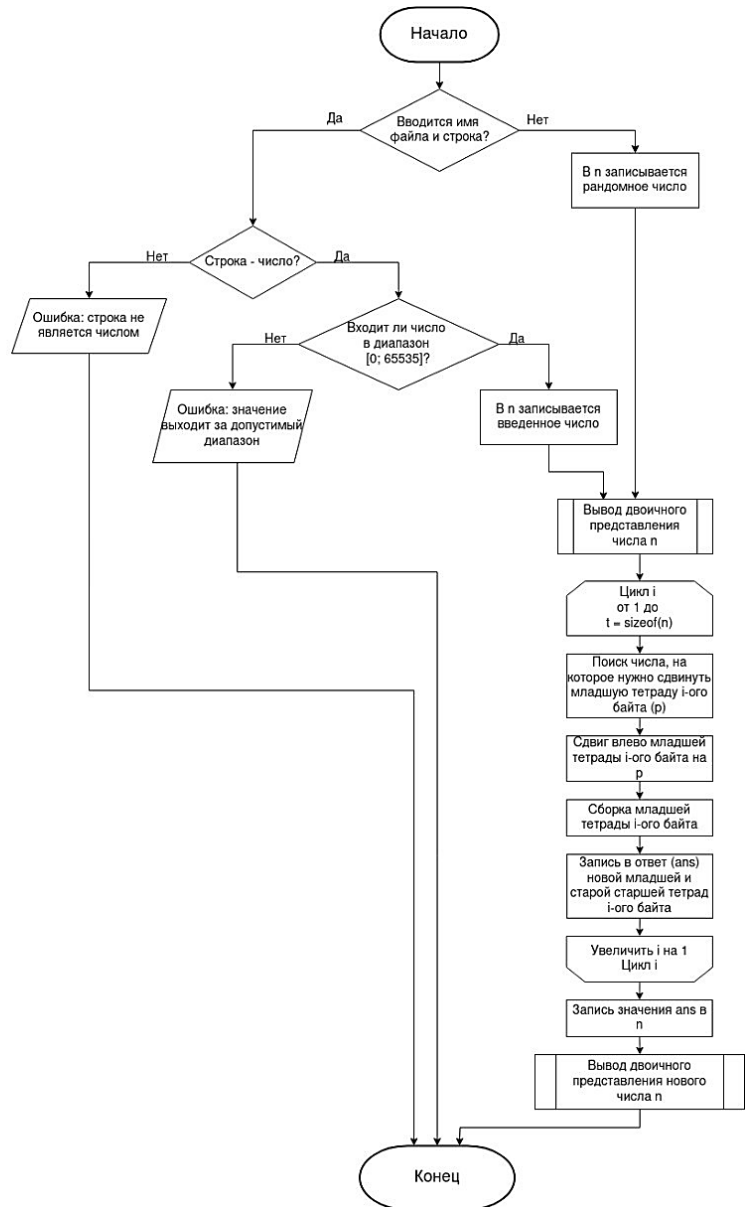
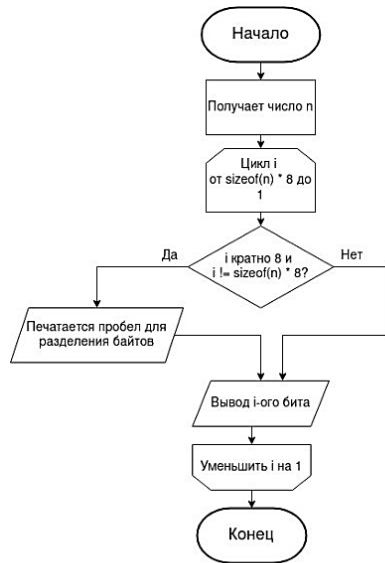
```
1 .PHONY: all clean
2
3 APP=lab1avsn3148
4 CFLAGS=-Wall -Wextra -Werror -g
5
6 all: $(APP)
7
8 $(APP):
9     gcc -o $(APP) $(CFLAGS) $(APP).c
10
11 clean:
12     rm $(APP)
13
```

Примеры работы программы на различных исходных данных



```
user@user-HP-Laptop-15s-eq3xxx: ~
user@user-HP-Laptop-15s-eq3xxx:~$ make all
gcc -o lab1avsn3148 -Wall -Wextra -Werror -g lab1avsn3148.c
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab1avsn3148
17233
01000011 01010001
01000110 01010010
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab1avsn3148 123
00000000 01111011
00000000 01111011
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab1avsn3148 220v
Ошибка: '220v' не является числом.
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab1avsn3148 home
Ошибка: 'home' не является числом.
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab1avsn3148 дом
Ошибка: 'дом' не является числом.
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab1avsn3148 70000
Ошибка: значение выходит за допустимый диапазон.
user@user-HP-Laptop-15s-eq3xxx:~$
```

Блок-схема алгоритма преобразования



Исходный текст программы с комментариями

```
#include <stdio.h>
#include <stdlib.h> //Здесь содержится функция rand()
#include <time.h> //Чтобы использовать функцию time()
#include <string.h> //Для NULL

void perevod (unsigned short n) { // Функция вывода двоичного числа побайтно
    for (int i = sizeof(n) * 8; i > 0; i--) {
        if (i % 8 == 0 && i != sizeof(n) * 8)
            printf(" ");
        printf("%d", n & (1 << (i - 1)) ? 1 : 0);
    }
    printf("\n");
}

int main(int argc, char *argv[]) {
    // Проверка запуска с переменной среды, включающей отладочный вывод.
    // Пример запуска с установкой переменной LAB1DEBUG в 1:
    // $ LAB1DEBUG=1 ./lab1avsN3148 123
    char *DEBUG = getenv("LAB1DEBUG");
    if (DEBUG) {
        fprintf(stderr, "Включен вывод отладочных сообщений\n");
    }

    /*
    if (argc != 2) {
        fprintf(stderr, "Usage: %s [число]\n", argv[0]);
        return EXIT_FAILURE;
    }
    */

    unsigned short n;
    long long k;
    char errno;
    int flag = 0;

    srand(time(NULL)); //Для генерации рандомных чисел

    //Обработка ошибок: проверка на корректность входных данных
    if (argc == 1) {
        n = rand();
        printf("%hu\n", n);
        flag = 1;
    }
    else if (argc == 2) {

        if (sscanf(argv[1], "%lld %c", &k, &errno) != 1)
            printf("Ошибка: \"%s\" не является числом.\n", argv[1]);

        else if (k < 0 || k > 65535)
            printf("Ошибка: значение выходит за допустимый диапазон.\n");

        else {
            n = k;
            flag = 1;
        }
    }
}
```

```

    }

    if (flag) { //Если входные данные корректны, можно переходить к выполнению программы
        perevod(n);

        int t, p;
        t = sizeof(n);
        unsigned short n1, n2, n3;
        unsigned short ans = 0; //Здесь будет лежать ответ

        unsigned short mask1 = 0b11000000, mask2 = 0b11110000, mask3 = 0b1111, mask4 =
0b11110000;

        for (int i = 1; i <= t; i++) {

            p = n & mask1;
            mask1 <<= 8; //Обновили mask1
            p >>= 6 + (i - 1) * 8; //Нашли число, на которое нужно двигать младшую тетраду i-
ого байта

            n1 = n & mask3; //Находим младшую тетраду i-ого байта
            n1 <<= p; //Делаем сдвиг младшей тетрады
            n2 = n1 & mask2;
            n1 = n1 & mask3;
            n2 >>= 4;
            n3 = n1 | n2; //Получили новую тетраду: соединяем две части новой младшей
тетрады

            mask2 <<= 8; //Обновили mask2
            mask3 <<= 8; //Обновили mask3

            ans |= n3; //Записали в ответ новую младшую тетраду i-ого байта
            ans |= n & mask4; //Записали в ответ старую старшую тетраду i-ого байта
            mask4 <<= 8; //Обновили mask4

        }

        n = ans;
        perevod(n);
    }

    return EXIT_SUCCESS;
}

```